مرور کامل پروژه Movie App

نسخهٔ ۲، بعد از تکمیل پروژه

React · Vite · React Router · Context · useEffect · TMDB API · CSS

> **تفاوت با نسخهٔ اول:** در نسخهٔ اول، صفحهٔ علاقه‌مندی‌ها خالی بود و چند باگ در کد وجود داشت. در این نسخه همهٔ آن‌ها رفع شده‌اند و پروژه کامل است. بخش ۸ نشان می‌دهد چه چیزی از نسخهٔ قبل تغییر کرد و چرا.

فهرست

1. نقشهٔ کلی و جریان اجرا
2. بررسی فایل به فایل
3. مفاهیم React که استفاده شد
4. مفاهیم JavaScript که استفاده شد
5. مفاهیم CSS که استفاده شد
6. منطق برنامه به زبان الگوریتم
7. باگ‌های رفع‌شده (قبل و بعد)
8. موارد باقی‌مانده و پیشنهاد بعدی

9. سؤال‌های مرور با پاسخ
10. واژه‌نامه

۱. نقشهٔ کلی و جریان اجرا

برنامه فیلم‌های محبوب را از سایت TMDB می‌گیرد و نشان می‌دهد. کاربر می‌تواند فیلم جستجو کند، با دکمهٔ ♥ آن را به علاقه‌مندی‌ها اضافه یا از آن حذف کند و لیست علاقه‌مندی‌ها را در صفحهٔ جدا ببیند. این لیست در مرورگر ذخیره می‌شود و با بستن و باز کردن سایت از بین نمی‌رود.

ساختار فایل‌ها

```
project/
├── index.html            تنها صفحهٔ HTML، شامل div#root
├── .env                  کلید API (در git قرار نمی‌گیرد)
├── .env.example          نمونهٔ .env بدون کلید واقعی
├── package.json
└── src/
    ├── main.jsx          نقطهٔ شروع: Router + StrictMode
    ├── App.jsx           Provider + NavBar + مسیرها
    ├── context/
    │   └── MovieContext.jsx   حافظهٔ مشترک علاقه‌مندی‌ها
    ├── components/
    │   ├── NavBar.jsx
    │   └── MovieCard.jsx
    ├── pages/
    │   ├── Home.jsx
    │   └── Favorites.jsx
    ├── services/
    │   └── api.js        ارتباط با TMDB
    └── css/              استایل‌ها
```

لایه	نقش
`index.html`	ظرف خالی که React داخلش برنامه را می‌سازد
`main.jsx`	روشن کردن برنامه
`App.jsx`	اسکلت: Provider، نوار بالا و تعیین صفحه بر اساس آدرس
`/context`	داده‌ای که چند صفحه به آن نیاز دارند (علاقه‌مندی‌ها)
`/pages`	هر صفحهٔ کامل
`/components`	تکه‌های قابل استفادهٔ مجدد
`/services`	صحبت با دنیای بیرون (API)
`/css`	ظاهر

جریان اجرا از لحظهٔ باز شدن سایت

1. مرورگر `index.html` را باز می‌کند.
2. تگ `script` فایل `main.jsx` را اجرا می‌کند.
3. `main.jsx` کامپوننت `App` را داخل `div#root` می‌گذارد، بین `StrictMode` و `BrowserRouter`.
4. `App` اجرا می‌شود. `MovieProvider` اول لیست علاقه‌مندی‌ها را از `localStorage` می‌خواند.
5. `NavBar` همیشه نمایش داده می‌شود. `Routes` آدرس را با `Route`ها مقایسه می‌کند: آدرس `/` صفحهٔ Home و آدرس `favorites/` صفحهٔ Favorites.
6. Home بعد از اولین نمایش، فیلم‌های محبوب را از API می‌گیرد.
7. هر بار state عوض شود، React صفحه را دوباره می‌سازد.

۲. بررسی فایل به فایل

۲-۱. index.html

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

`div#root` خالی است چون React محتوای برنامه را با جاوااسکریپت داخلش می‌سازد. `"type="module` یعنی فایل می‌تواند `import` و `export` داشته باشد.

۲-۲. main.jsx

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter> <App /> </BrowserRouter>
  </StrictMode>,
)
```

- `createRoot` عنصر `root` را به React می‌دهد.
- `StrictMode` فقط در حالت توسعه اجرا می‌شود و مشکلات پنهان را با اجرای دوبارهٔ effectها نشان می‌دهد.
- `BrowserRouter` باید بیرون `App` باشد، چون `Routes` و `Link` داخل `App` به آدرس فعلی نیاز دارند.

۲-۳. App.jsx

```
<MovieProvider>
  <NavBar />
  <main className="main-content">
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/favorites" element={<Favorites />} />
    </Routes>
  </main>
</MovieProvider>
```

- `MovieProvider` همهٔ برنامه را در بر می‌گیرد تا هر کامپوننت داخلش به علاقه‌مندی‌ها دسترسی داشته باشد.
- `NavBar` بیرون `Routes` است، پس در همهٔ صفحه‌ها ثابت می‌ماند و فقط `main` عوض می‌شود.

۲-۴. context/MovieContext.jsx (جدید)

```
const MovieContext = createContext()
export const useMovieContext = () => useContext(MovieContext)

export const MovieProvider = ({ children }) => {
  const [favorites, setFavorites] = useState(() => {
    const storedFavs = localStorage.getItem("favorites")
    return storedFavs ? JSON.parse(storedFavs) : []
  })

  useEffect(() => {
    localStorage.setItem('favorites', JSON.stringify(favorites))
  }, [favorites])

  const addToFavorites = (movie) =>
    setFavorites(prev => [...prev, movie])
  const removeFromFavorites = (movieId) =>
    setFavorites(prev => prev.filter(m => m.id !== movieId))
  const isFavorite = (movieId) =>
    favorites.some(m => m.id === movieId)

  const value = { favorites, addToFavorites, removeFromFavorites, isFavorite }
  return <MovieContext.Provider value={value}>{children}</MovieContext.Provider>
}
```

منطق:

1. `useState` با تابع شروع می‌شود (lazy initial state): فقط یک بار، هنگام اولین رندر، از `localStorage` می‌خواند. اگر چیزی ذخیره نشده بود، آرایهٔ خالی می‌گذارد.
2. هر بار `favorites` عوض شود، effect دوم آن را با `JSON.stringify` ذخیره می‌کند. `localStorage` فقط متن نگه می‌دارد، پس تبدیل لازم است.
3. `setFavorites(prev => ...)` از نسخهٔ قبلی state استفاده می‌کند و آرایهٔ جدید می‌سازد. آرایه را مستقیم تغییر نمی‌دهیم (immutability).
4. مقدار `value` به Provider داده می‌شود. اگر `value` نباشد، `useMovieContext()` مقدار `undefined` برمی‌گرداند و برنامه می‌شکند.

> **چرا Context؟** هر دو صفحه (Home و Favorites) و کامپوننت `MovieCard` به یک لیست نیاز دارند. بدون Context باید state را در `App` بسازیم و چند لایه props پاس بدهیم (prop drilling).

۲-۵. components/NavBar.jsx

دو `Link` به `/` و `Link / favorites` آدرس را بدون بارگیری مجدد صفحه عوض می‌کند، برخلاف تگ `a`.

۲-۶. components/MovieCard.jsx

```
const { isFavorite, addToFavorites, removeFromFavorites } = useMovieContext();
const favorite = isFavorite(movie.id);

function onFavoriteClick(e) {
  e.preventDefault();
  if (favorite) removeFromFavorites(movie.id);
  else addToFavorites(movie);
}
```

- کارت `movie` را از props می‌گیرد. وضعیت علاقه‌مندی را از Context می‌پرسد.
- با کلیک، فیلم اگر علاقه‌مندی باشد حذف و در غیر این صورت اضافه می‌شود. بعد Context عوض می‌شود و همهٔ کارت‌ها دوباره رندر می‌شوند، پس رنگ ♥ خودکار عوض می‌شود.
- کلاس `active` فقط وقتی `favorite` درست باشد اضافه می‌شود: `favorite-btn ${favorite ? "active" : ""}`.
- اگر `poster_path` نداشته باشد، به‌جای عکس خراب، یک جعبهٔ «No poster» نشان داده می‌شود.
- `movie.release_date?.split("-")[0]` فقط سال را برمی‌دارد. علامت `?.` از خطا هنگام نبودن تاریخ جلوگیری می‌کند.

۲-۷. pages/Home.jsx

چهار state دارد:

state	مقدار اولیه	کار
`searchQuery`	`""`	متن داخل input
`movies`	`[]`	فیلم‌های نمایش‌داده‌شده
`error`	`null`	پیام خطا
`loading`	`true`	در حال بارگیری

```
useEffect(() => {
  const loadPopularMovies = async () => {
    try {
      setMovies(await getPopularMovies());
    } catch (err) {
      console.error(err);
      setError("Failed to load Movies...");
    } finally {
      setLoading(false);
    }
  };
  loadPopularMovies();
}, []);
```

- آرایهٔ وابستگی `[]` یعنی effect فقط یک بار، بعد از اولین نمایش اجرا شود.
- `finally` تضمین می‌کند `loading` چه موفق چه ناموفق خاموش شود.

```
const handleSearch = async (e) => {
  e.preventDefault();
  if (!searchQuery.trim()) return;
  if (loading) return;
  setLoading(true);
  setError(null);
  try {
    setMovies(await searchMovies(searchQuery));
  } catch (err) {
    console.error(err);
    setError("Failed to search movies...");
  } finally {
    setLoading(false);
  }
};
```

چهار حالت نمایش:

loading	error	movies	صفحه چه نشان می‌دهد
true	هر چیزی	هر چیزی	«Loading...»
false	null	پر	شبکهٔ فیلم‌ها
false	پیام	هر چیزی	پیام خطا (و شبکهٔ فعلی)
false	null	خالی	«No movies found» (جدید)

۸-۲. pages/Favorites.jsx

```
const { favorites } = useMovieContext();
if (favorites.length > 0) { /* شبکهٔ کارت‌ها */ }
return /* پیام «No Favorite Movie Yet» */;
```

از همان `MovieCard` استفاده می‌کند. چون وضعیت در Context است، با زدن ♥ روی یک کارت در این صفحه، همان لحظه از لیست حذف می‌شود.

۹-۲. services/api.js

```
const API_KEY = import.meta.env.VITE_TMDB_KEY;
const BASE_URL = "https://api.themoviedb.org/3";

const fetchMovies = async (url) => {
  const response = await fetch(url);
  if (!response.ok) throw new Error("Request failed: " + response.status);
  const data = await response.json();
  return data.results ?? [];
};

export const getPopularMovies = () =>
  fetchMovies(`${BASE_URL}/movie/popular?api_key=${API_KEY}`);
export const searchMovies = (query) =>
  fetchMovies(`${BASE_URL}/search/movie?
api_key=${API_KEY}&query=${encodeURIComponent(query)}`);
```

- کد fetch در فایل جدا است تا صفحه‌ها فقط با داده کار کنند و اگر API عوض شد، فقط یک فایل عوض شود.
- `fetch` برای ۴۰۱ یا ۵۰۰ خطا نمی‌دهد. به همین دلیل `response.ok` دستی چک می‌شود.
- `data.results ?? []` اگر نتیجه نبود، آرایهٔ خالی برمی‌گرداند تا `movies.map` نشکند.
- `encodeURIComponent` نویسه‌های خاص متن جستجو (فاصله، `&`) را برای آدرس ایمن می‌کند.

۱۰-۲. فایل .env

متغیرهای محیطی Vite باید با `VITE_` شروع شوند تا در کد قابل خواندن باشند. فایل `.env` در `.gitignore` است تا کلید روی GitHub نرود. بعد از تغییر `.env` سرور dev خودش دوباره راه‌اندازی می‌شود.

> **هشدار:** هر چیزی که در کد فرانت‌اند باشد، برای کاربر قابل دیدن است (حتی از `.env` ، چون در فایل ساخته‌شده قرار می‌گیرد). کلید TMDB برای این پروژه مشکلی ندارد، ولی برای کلیدهای حساس به بک‌اند نیاز دارید.

۳. مفاهیم React که استفاده شد

مفهوم	توضیح	کجا
کامپوننت	تابعی که JSX برمی‌گرداند و می‌توان چندبار از آن استفاده کرد	همهٔ فایل‌ها
JSX	نوشتن شبیه HTML داخل جاوااسکریپت. مرورگر نمی‌فهمد و Vite تبدیلش می‌کند	همهٔ فایل‌ها
props	ورودی کامپوننت از والد. فقط خواندنی	`MovieCard ...=movie`
state	حافظهٔ کامپوننت. تغییرش رندر مجدد می‌سازد	Home، MovieContext
useEffect	کار جانبی بعد از رندر (fetch، ذخیره)	Home، MovieContext
Context	اشتراک داده بین کامپوننت‌ها بدون props	MovieContext
Router	نمایش صفحه بر اساس آدرس	main.jsx، App.jsx
Controlled input	مقدار input از state می‌آید و با `onChange` به‌روز می‌شود	فرم جستجو
Conditional rendering	نمایش شرطی با `&&` و `? :`	Home، Favorites
لیست و key	`map` برای ساخت کارت‌ها. `key` یعنی شناسهٔ یکتا برای هر آیتم	`key={movie.id}`
Custom hook	تابع `useMovieContext` که `useContext` را پنهان می‌کند	MovieContext

> **قانون state:** state را مستقیم تغییر نده. همیشه تابع set را صدا بزن و برای آرایه‌ها نسخهٔ جدید بساز (`[prev, movie...]` و `filter`). React فقط با صدا زدن تابع set متوجه تغییر می‌شود.

۴. مفاهیم JavaScript که استفاده شد

مفهوم	مثال از پروژه
import / export	`export const getPopularMovies` و `import { getPopularMovies }` . با export نام‌دار، اسم باید دقیقاً یکی باشد.
async / await	`await fetch(...)` داخل تابع `async`
try / catch / finally	مدیریت خطا و خاموش کردن loading
Destructuring	`const { favorites } = useMovieContext()`
Spread	`[prev, movie...]`
filter / some / map	حذف، پرسیدن وجود، ساخت کارت
Optional chaining `.?`	`movie.release_date?.split("-")[0]`
Nullish coalescing `??`	`data.results ?? []`
Template literal	`` `movie/popular/${BASE_URL}` ``
JSON	`JSON.stringify` و `JSON.parse` برای `localStorage`
localStorage	ذخیرهٔ متن در مرورگر که با بستن سایت از بین نمی‌رود

۵. مفاهیم CSS که استفاده شد

- **Flexbox:** چیدمان نوار بالا و فرم جستجو (`display: flex`).
- **Grid:** `grid-template-columns: repeat(auto-fit, minmax(300px, 1fr))` هر ستون حداقل ۳۰۰ پیکسل است، به تعداد جا ستون می‌سازد و باقی فضا را مساوی تقسیم می‌کند.
- **aspect-ratio:** `2/3` برای پوستر، تا اندازهٔ کارت‌ها یکدست باشد.
- **position relative / absolute:** دکمهٔ ♥ روی پوستر قرار می‌گیرد.
- **Media query:** `@media (max-width: 768px)` برای موبایل.
- **@keyframes:** انیمیشن `fadeIn` برای کارت‌ها.
- **تم تیره و روشن:** `prefers-color-scheme` در `index.css` .

> **خطای رایج:** اسم کلاس در JSX باید دقیقاً با CSS یکی باشد. در نسخهٔ اول، `favorites.` روی تگ اشتباه بود و استایل اعمال نمی‌شد. حالا wrapper این کلاس را دارد.

۶. منطق برنامه به زبان الگوریتم

الف) باز شدن سایت

1. مرورگر `index.html` را می‌گیرد و `main.jsx` را اجرا می‌کند.
2. Provider لیست علاقه‌مندی‌ها را از `localStorage` می‌خواند.
3. Router آدرس را می‌خواند و صفحه را انتخاب می‌کند.
4. اگر صفحه Home است: `loading=true` ، سپس فیلم‌های محبوب را بگیر. موفق: `movies` را پر کن. خطا: پیام بگذار. در هر دو حالت `loading=false` .

ب) جستجو

1. کاربر تایپ می‌کند و `searchQuery` با هر حرف به‌روز می‌شود.
2. کاربر Enter یا دکمه را می‌زند و `handleSearch` اجرا می‌شود.
3. اگر متن خالی یا در حال بارگیری است: خروج.
4. `loading=true` و `error=null` . نتیجه را از API بگیر و `movies` را عوض کن. خطا: `error` را بگذار. در پایان `loading=false` .

ج) علاقه‌مندی

1. کاربر ♥ را روی کارت می‌زند.
2. اگر فیلم در لیست است، حذفش کن؛ در غیر این صورت اضافه‌اش کن.
3. `favorites` عوض می‌شود، پس همهٔ کارت‌های وابسته دوباره رندر می‌شوند.
4. effect لیست جدید را در `localStorage` ذخیره می‌کند.
5. با ریلود، مرحلهٔ ۲ بخش (الف) لیست را برمی‌گرداند.

د) ناوبری

1. کاربر روی Favorites کلیک می‌کند و `Link` جلوی بارگیری مجدد را می‌گیرد.
2. آدرس به `favorites/` عوض می‌شود و `Routes` صفحهٔ Favorites را نشان می‌دهد.
3. NavBar تغییر نمی‌کند، چون بیرون `Routes` است.

> **نکتهٔ جالب:** در نسخهٔ اول، وقتی از Home به Favorites و برمی‌گشتی، متن جستجو پاک می‌شد چون state صفحه با خارج شدن از بین می‌رود. این هنوز هم صادق است: متن جستجو و نتیجهٔ جستجو state محلی Home هستند. علاقه‌مندی‌ها چون در Context هستند، از بین نمی‌روند.

۷. باگ‌های رفع‌شده (قبل و بعد)

۱. غلط املایی satError (Home.jsx)

```
// قبل
satError("Failed to search movies...");
// بعد
setError("Failed to search movies...");
```

قبل از رفع، هنگام خطا یک `ReferenceError` می‌آمد و کاربر پیامی نمی‌دید. **درس:** غلط‌های املایی در اسم متغیر شایع‌ترین خطا هستند. از F2 برای تغییر نام و از lint استفاده کن.

۲. چک نشدن response.ok (api.js)

```
// قبل
const data = await response.json();
return data.results;
// بعد
if (!response.ok) throw new Error("Request failed: " + response.status);
return data.results ?? [];
```

fetch برای ۴۰۱ یا ۵۰۰ خطا نمی‌داد. `data.results` می‌شد `undefined` و `movies.map` صفحه را سفید می‌کرد.

۳. کلید API داخل کد

قبل: `const API_Key = "..."` داخل `api.js`. بعد: `import.meta.env.VITE_TMDB_KEY` و فایل `.env` که در `.gitignore` است. اسم ثابت هم به `API_KEY` تغییر کرد.

> چون کلید قبلاً در کد و تاریخچهٔ git بوده، اگر پروژه را عمومی کردی، در TMDB یک کلید جدید بساز.

۴. صفحهٔ Favorites همیشه لیست نشان می‌داد

```
// قبل: آرایه (حتی خالی) همیشه true است
if (favorites) { ... }
// بعد
if (favorites.length > 0) { ... }
```

۵. خطای قدیمی پاک نمی‌شد

`setError(null)` از بعد از موفقیت به ابتدای جستجو رفت تا پیام قبلی هنگام درخواست جدید نماند.

۶. فیلم بدون پوستر

شرط روی `poster_path` اضافه شد تا به‌جای آدرس خراب (`w500null/...`)، جایگزین نشان داده شود.

۷. حالت خالی جستجو

اگر جستجو هیچ نتیجه‌ای نداشت، صفحهٔ خالی دیده می‌شد. حالا «No movies found» نمایش داده می‌شود.

۸. دکمهٔ علاقه‌مندی و Context

دکمه قبلاً فقط `alert` بود. حالا به Context وصل است. در `MovieContext.jsx` چند خطا هم رفع شد: نبود `=>` در تعریف تابع، اسم setter اشتباه (`setFavorite` به‌جای `setFavorites`)، assign شدن به setter به‌جای صدا زدن، `Provider` با حرف کوچک و نبود `value` .

۹. هشدار lint: setState در effect

خواندن از `localStorage` از effect به `useState(() => ...)` منتقل شد تا رندر اضافه ایجاد نشود.

۱۰. تمیزکاری

`console.log` برای خطا به `console.error` تغییر کرد و `import React` اضافه در NavBar حذف شد.

۸. موارد باقی‌مانده و پیشنهاد بعدی

مورد	توضیح
هشدار only-export-components	`MovieContext.jsx` هم هوک و هم کامپوننت export می‌کند. فقط روی Fast Refresh اثر دارد. راه‌حل: هوک را به فایل جدا ببر.
صفحهٔ ۴۰۴	هنوز `path="*"` Route نداریم. آدرس نامعتبر صفحهٔ خالی نشان می‌دهد.
حفظ جستجو	متن جستجو با رفتن به صفحهٔ دیگر پاک می‌شود. برای نگه داشتنش، state را به Context ببر.
صفحه‌بندی	API فقط صفحهٔ اول نتایج را برمی‌گرداند. پارامتر `page` را می‌توان اضافه کرد.
debounce	جستجو فقط با Enter است. جستجوی زنده نیازمند تأخیر کوتاه برای کم کردن درخواست‌هاست.
تست	برای `api.js` و `MovieContext` می‌توان تست نوشت.
Deploy	برای انتشار روی Netlify یا Vercel، متغیر `VITE_TMDB_KEY` را در تنظیمات همان سرویس بگذار.

۹. سؤال‌های مرور با پاسخ

1. **چرا `div#root` خالی است؟** چون React محتوای برنامه را با جاوااسکریپت داخلش می‌سازد.
2. **تفاوت Link و a؟** `Link` فقط آدرس را عوض می‌کند و React محتوا را عوض می‌کند. `a` کل صفحه را از سرور دوباره می‌گیرد.
3. **چرا BrowserRouter بیرون App است؟** چون `Link` و `Routes` داخل `App` به Router نیاز دارند.
4. **props و state؟** props ورودی از والد و فقط خواندنی است. state حافظهٔ خود کامپوننت است و تغییرش رندر مجدد می‌سازد.
5. **چرا `useState([])` و نه `null`؟** `[].map` کار می‌کند ولی `null.map` خطا می‌دهد.
6. **آرایهٔ `[]` در useEffect؟** effect فقط یک بار بعد از اولین نمایش اجرا شود.
7. **چرا `setLoading(false)` در finally؟** تا چه موفق چه ناموفق، loading خاموش شود.
8. **چرا `e.preventDefault()`؟** فرم نباید صفحه را reload کند، چون state از دست می‌رود.
9. **key برای چیست؟** تا React هر آیتم لیست را شناسایی و تغییرات را بهینه اعمال کند.
10. **چرا نباید state را مستقیم عوض کرد؟** React فقط با صدا زدن set متوجه تغییر می‌شود.
11. **چرا Context؟** برای اینکه Favorites، Home و MovieCard لیست مشترک داشته باشند، بدون prop drilling.
12. **چرا `JSON.stringify`؟** چون `localStorage` فقط متن نگه می‌دارد.
13. **چرا state اولیه تابع است؟** تا خواندن از `localStorage` فقط یک بار، در اولین رندر انجام شود، نه هر رندر.
14. **اگر fetch پاسخ ۴۰۱ بدهد چه می‌شود؟** خطا نمی‌دهد. به همین دلیل `response.ok` را چک می‌کنیم.
15. **چرا در StrictMode درخواست دو بار می‌رود؟** فقط در توسعه، برای نشان دادن مشکلات پنهان. در build نهایی یک بار است.
16. **چرا NavBar بیرون Routes است؟** تا در همهٔ صفحه‌ها ثابت بماند.
17. **چرا کد fetch در `api.js` است؟** برای جداسازی مسئولیت‌ها، استفادهٔ مجدد و تغییر راحت‌تر API.
18. **چرا `env.` باید `_VITE` داشته باشد؟** فقط متغیرهای با این پیشوند در کد فرانت‌اند در دسترس قرار می‌گیرند.

۱۰. واژه‌نامه

اصطلاح	معنی
Component	تابعی که JSX برمی‌گرداند
JSX	نوشتن شبیه HTML داخل جاوااسکریپت
Props	ورودی‌های کامپوننت از والد
State	حافظهٔ کامپوننت که تغییرش رندر مجدد می‌سازد
Hook	تابع ویژهٔ React که با `use` شروع می‌شود
Effect	کار جانبی بعد از رندر
Context	مکانیزم اشتراک داده بین کامپونت‌ها بدون props
Provider	کامپوننتی که مقدار Context را در اختیار فرزندانش می‌گذارد
Prop drilling	پاس دادن props از چند لایه فقط برای رساندن به فرزند عمیق
Lazy initial state	دادن تابع به `useState` تا فقط یک بار اجرا شود
Dependency array	فهرست مقادیری که تعیین می‌کنند effect کی اجرا شود
Controlled input	input که مقدارش از state می‌آید
Router	نمایش صفحه بر اساس آدرس
SPA	برنامهٔ تک‌صفحه‌ای بدون بارگیری مجدد کامل
Immutability	تغییر ندادن داده‌ی قبلی و ساختن نسخهٔ جدید
API / Endpoint	رابط ارتباط برنامه‌ها / یک آدرس مشخص از آن
JSON	قالب متنی برای تبادل داده
localStorage	حافظهٔ متنی مرورگر که ماندگار است
Environment variable	مقدار تنظیماتی خارج از کد، مثل کلید API
Empty state	نمایش مناسب وقتی داده‌ای نیست
Optional chaining	`?.` برای جلوگیری از خطا روی مقدار تعریف‌نشده
Bundler	ابزار تبدیل و فشرده‌سازی کد (Vite)

تمرین نهایی: بدون نگاه به این سند، مسیر «باز شدن سایت تا اضافه کردن یک فیلم به علاقه‌مندی‌ها و ماندنش بعد از ریلود» را با زبان خودت توضیح بده. بعد سؤال‌های بخش ۹ را جواب بده. هر جا گیر کردی، به همان بخش برگرد.